

Payment Processing Platform

Take-home submission documentation

Contents

Payment Processing Platform	1
Scope — where the depth went	1
Setup and running locally	2
Architectural overview	4
Tradeoffs	6
Assumptions	7
Testing	8
Production considerations	8
Future improvements	9

Payment Processing Platform

A simplified payment processing platform built for the senior engineer technical exercise: a .NET 10 API, a React dashboard, and an asynchronous settlement workflow backed by Postgres. This README is the whole written deliverable — setup, architecture, tradeoffs, assumptions, production considerations, and future work. The implementation plan it was built from is in [docs/PLAN.md](#), untouched since it was written.

The brief grades judgment rather than feature count, so the documentation spends most of its words on **why** things are the way they are, and on what is deliberately missing.

Scope — where the depth went

One payment flow, taken seriously end to end: create → authorize → capture → async settle → refund. The failure modes are treated as the feature — client retries, concurrent duplicates, crashed workers, refunds racing settlement, jobs that will never succeed — and each one has tests, several verified by mutation (break the mechanism, watch exactly the right tests fail).

Deliberately not built, each with its production path documented below: partial and multiple captures/refunds, multi-currency, real authentication (a header stub), a real acquirer (a simulated gateway), broker infrastructure (a transactional outbox instead), webhooks, automatic dead-letter replay.

Everything in the repo beyond that slice exists to make the slice *observable*, not to widen it: a demo profile seeds history and drives real traffic so the dashboard and metrics show true numbers, and an observability profile points a real Grafana/Prometheus/Loki/Tempo at the seams to prove they are seams. Both are optional compose profiles; the core neither knows nor cares whether they're running.

Setup and running locally

The whole stack, one command

Prerequisite: Docker (with Compose v2) — nothing else. No .NET SDK, no Node.

```
docker compose --profile demo up --build
```

This builds the API and the dashboard, waits for Postgres to be healthy, applies migrations, and starts the settlement worker.

- **Dashboard:** <http://localhost:5173> ← start here
- **API:** <http://localhost:5080> — redirects to Swagger, where every endpoint can be tried
- **Metrics:** <http://localhost:5080/metrics> — health: `/healthz`, `/readyz`

What you'll see. The profile seeds a week of history (~400 payments) into an empty database — creating a payment is an API-only operation, so without seed data the first view would be an empty table with no way to fill it. The history is shaped rather than random (busy weekdays, dead nights, lighter weekends, a long tail of small baskets), every seeded payment goes through the real aggregate so each has a genuine audit trail, and the seed is deterministic so every reviewer sees the same week.

A dozen or so payments are left `Authorized`. Open one, press **Capture**, and watch the timeline grow to `Settled` on its own a second later as the worker picks the job up — one click exercising the state machine, the idempotency key, the transactional outbox, the audit trail and the async worker together.

Two planted details worth knowing about: one payment is stuck `Captured` behind a **dead-lettered** settlement job (five attempts, `acquirer timeout`) — the 2am scenario, sitting in the data, queryable and alertable — and ~20% of successful settlements took a retry, so `settlement_jobs` looks like a real outbox rather than a clean one.

In this profile authorization is pinned to always approve so a walkthrough doesn't hit a random decline; settlement still fails ~20% of the time, so retries and backoff are visible in `docker compose logs api`. To see the decline path, use a card token ending in `-declined`, or override the rate:

```
FakeGateway__AuthorizeDeclineRate=0.5 docker compose --profile demo up --build
```

Tear down with `docker compose --profile demo down` (add `-v` to drop the database volume and let the demo re-seed).

Optional: Grafana, Prometheus, Loki, Tempo

```
docker compose --profile demo --profile observability up --build
```

- **Grafana:** <http://localhost:3000> — opens straight on the *Payments — overview* dashboard, no login (anonymous admin; local demo only)
- **Prometheus:** <http://localhost:9090>
- **Traces:** Tempo, inside Grafana (**Drilldown** → **Traces**, or click the **TraceID** button on any log line for that request's span waterfall)

Everything is provisioned — datasources, the dashboard, the scrape config. The dashboard shows the settlement queue (depth, lag, dead-letter backlog), the payment lifecycle, RED, and a log panel with a search box: paste a correlation id and you get one payment's whole journey, including the worker settling it seconds later on another thread.

The numbers on those graphs are earned, not backfilled. The demo profile runs a load generator making real HTTP requests at the API (~10 payments/min): authorized by the fake gateway, captured, settled by the real worker, some declined, some refunded, some retried with the same idempotency key like a flaky client would. The seeder deliberately never touches the metrics — counters live in the request path, so Prometheus only shows traffic that actually happened. That's also why **the Grafana time range should be short (15-30 minutes)**: a seven-day window on a

stack that booted five minutes ago is empty, correctly. Two artifacts of this design are visible on the dashboard and are intended: requests/min and payment events/min disagree (the generator retries ~15% of creates; the server replays them, so RED counts a request and payments_created_total doesn't count a payment), and `Dead-lettered jobs: 1` is the seeder's planted stuck payment giving the alert something real to fire on.

The application does not change to make any of this work: it writes JSON to stdout and exposes /metrics exactly as it does without the profile. Alloy ships container stdout to Loki; Prometheus scrapes the endpoint that was already there; Tempo receives the OTLP traces that were already being emitted. That is the “seams, not a stack” tradeoff (below) demonstrated rather than claimed.

The dev loop (hot reload)

Prerequisites: **.NET 10 SDK**, **Node 22** (what CI runs; Vite 8 requires 20.19+ or 22.12+), **Docker**.

```
docker compose up -d                                # Postgres only; demo services stay out
cd backend && dotnet run --project src/Payments.Api    # API on :5080, migrates on startup
cd frontend && npm install && npm run dev             # dashboard on :5173 (Vite proxies /api)
```

Postgres is published on host port **5433** to avoid clashing with a local Postgres on 5432.

The two paths use the same ports, so run one or the other. If a host dev server and the demo stack are both up, localhost may resolve to whichever bound IPv6 first — stop one before starting the other.

Outside the demo profile the fake gateway declines ~15% of authorizations and fails ~20% of settlements, so the failure paths are demonstrable by hand. For a deterministic run:

```
FakeGateway__AuthorizeDeclineRate=0 FakeGateway__SettleFailureRate=0 \
dotnet run --project src/Payments.Api
```

Tests

```
cd backend && dotnet test                            # domain + integration; Docker must be running (Testcontainers)
cd frontend && npm test                               # vitest
```

The integration tests start their own throwaway Postgres via Testcontainers — they never touch the compose database. CI ([github/workflows/ci.yml](https://github.com/workflows/ci.yml)) runs both suites plus frontend lint and a type-checked build on every push.

Configuration

There are **no required environment variables**: defaults in appsettings.json point at the compose Postgres, and the demo profile sets everything it needs in docker-compose.yml. Everything is overridable with standard .NET Section_Key syntax. The ones worth knowing:

Variable	Default	Effect
ConnectionStrings__Payments	localhost:5433, user/pass payments/payments_dev	Postgres connection
ASPNETCORE_ENVIRONMENT	Development (local)	Development applies migrations on startup and serves Swagger
FakeGateway__AuthorizeDeclineRate	0.15 (0 in demo)	Chance an authorization declines
FakeGateway__SettleFailureRate	0.2	Chance a settlement attempt fails transiently
Settlement__Enabled	true	Turn the in-process worker off (tests do)

Variable	Default	Effect
Settlement__PollInterval / BatchSize / MaxAttempts / BaseDelay / MaxDelay / LeaseDuration	2s / 10 / 5 / 2s / 5m / 1m	Worker tuning
Demo__Seed / Demo__Days / Demo__PaymentsPerDay	false / 7 / 70	Seeder (demo profile turns it on)
DemoTraffic__Enabled / DemoTraffic__PaymentsPerMinute	false / 10	Load generator (demo profile turns it on)
OTEL_EXPORTER_OTLP_ENDPOINT	unset	When set, registers the OTLP trace exporter; unset, tracing stays a no-op

The compose credentials are plain text because they are local-only and never a real secret.

Inspecting the database

postgresql://payments:payments_dev@localhost:5433/payments

Paste that into any client, or use the container's psql:

docker compose exec postgres psql -U payments -d payments

Four tables, and three queries that show what the system is actually about:

```
-- The audit trail for one payment. The Settled row's actor is settlement-worker, and its
-- correlation_id matches the Captured row's - the worker read it back off the job it
-- claimed, so the async leg is traceable to the request that caused it.
SELECT from_status, to_status, actor, correlation_id, occurred_at
FROM payment_transitions WHERE payment_id = '<id>' ORDER BY occurred_at;

-- The outbox. Written in the same transaction as the Captured transition, drained with
-- FOR UPDATE SKIP LOCKED. last_error and attempt_count are the retry story; a Dead row is
-- a payment parked for an operator.
SELECT status, attempt_count, next_attempt_at, last_error, correlation_id FROM settlement_jobs;

-- Stored idempotency responses. response_body is replayed verbatim on a retry;
-- request_hash is what makes a reused key with a different payload a 409.
SELECT operation, key, request_hash, response_status_code FROM idempotency_keys;
```

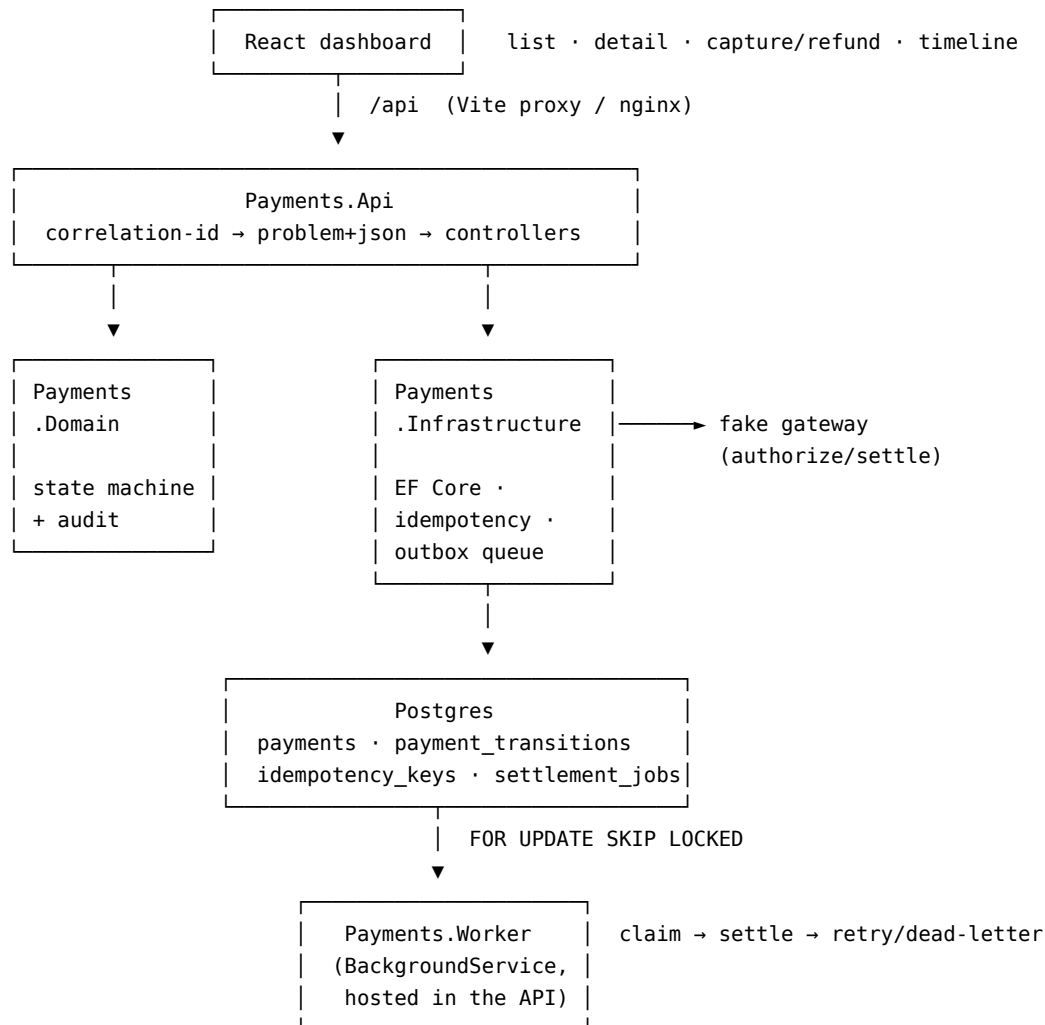
Data lives in the pgdata volume: it survives docker compose down but not down -v.

Architectural overview

Four projects with one-way dependencies: Domain ← Infrastructure ← Api, with Worker beside the API rather than inside it.

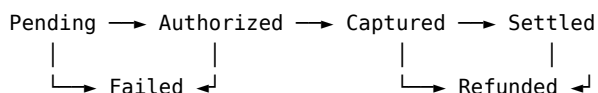
Project	Responsibility
Payments.Domain	Payment aggregate + state machine. Zero dependencies.
Payments.Infrastructure	EF Core persistence, idempotency store, settlement queue (outbox-style), fake gateway, metrics.
Payments.Worker	Settlement BackgroundService — claims jobs, retries with backoff, dead-letters.

Project	Responsibility
Payments.Api	HTTP endpoints, middleware (correlation id, problem+json), composition root.



The worker is hosted in the API process today, but the only coupling is two lines in `Program.cs` — promoting it to its own deployment is a hosting change, not a rewrite. This shape is deliberate: a modular monolith is right-sized for one domain slice, and the seams a reviewer would ask about are visible as project references without the distributed-systems tax.

Payment lifecycle



Failed and **Refunded** are terminal; **Refunded** is reachable from **Captured** *or* **Settled** — refunding after the money has moved is the real-world case, not an edge case.

`PaymentStateMachine` holds the transition table and is the single source of truth. Every mutation goes through one private `Transition()` method on the aggregate, which is also what appends the audit record — so an illegal transition and an unaudited transition are both impossible by construction rather than by review. The domain tests assert the full 36-pair matrix.

Request and data flow

Create authorizes synchronously at the gateway (auth-then-capture); capture enqueues a `settlement_jobs` row **in the same transaction** as the `Captured` transition; the worker polls that table with `FOR UPDATE SKIP LOCKED`, calls the gateway to settle, and moves the payment to `Settled` — saving the payment, its audit transition and the job's completion together, so a job cannot be marked done unless the payment really settled. The correlation id from the capture request is persisted on the job row and re-established in the worker's log scope, which is what makes one payment's journey greppable across the async boundary.

API

POST	/api/payments	Idempotency-Key required
GET	/api/payments?status=&from=&to=&search=&page=&pageSize=	
GET	/api/payments/{id}	
GET	/api/payments/{id}/transitions	audit trail; feeds the UI timeline
POST	/api/payments/{id}/capture	Idempotency-Key required
POST	/api/payments/{id}/refund	Idempotency-Key required
GET	/healthz /readyz /metrics	

Errors are RFC 7807 `application/problem+json` with a stable `errorCode` extension (`idempotency_conflict`, `invalid_state_transition`, `payment_not_found`, ...) so clients branch on one field rather than parse prose. Validation is 400, missing merchant identity is 401, unknown payment is 404; illegal transitions, key conflicts and lost concurrency races are 409.

Merchant identity comes from an `X-Merchant-Id` header (the auth stub) and **every** query is scoped by it at the repository boundary. Another merchant's payment is a 404, not a 403 — you shouldn't be able to learn that someone else's payment id exists. The worker uses a separate, deliberately unscoped lookup, kept apart so the scoping rule on API-facing queries stays impossible to forget.

The API is deliberately unversioned: with exactly one consumer, which lives in this repo and deploys with it, a version segment is ceremony. The cheap time to add `/v1` is immediately before the first external consumer appears; the wrong time is after.

Tradeoffs

A Postgres table as the queue, not a broker. The capture handler writes a `settlement_jobs` row in the *same transaction* as the `Captured` transition, and the worker claims rows with `FOR UPDATE SKIP LOCKED`. The obvious alternative — commit the capture, then publish to SQS/Rabbit — is a dual-write: the publish can fail after the capture commits and the settlement is silently lost. The outbox makes that impossible with zero extra infrastructure, gives at-least-once delivery, and makes queue-depth metrics a `count(*)`. *Given up*: independent scaling, push semantics, and a latency floor set by the poll interval. *Production*: the outbox stays exactly as it is; a relay publishes from it to a broker and the worker becomes its own deployment. This design is a step on that path, not a detour.

Idempotency reserves the key and does the work in one transaction. The stored response, the payment's state change, its audit rows and the settlement job all commit together, so a key can never be consumed without its side effects landing, or vice versa. A concurrent duplicate blocks on the key's unique index until the first request commits, then replays it — no in-flight state is ever observable, and there is no reservation lease to expire and reclaim. *Given up*: the gateway round trip happens with a transaction open, and the real cost is worse than one pinned connection per in-flight payment — **each blocked duplicate holds a connection too**. Play out the exact scenario idempotency exists for: the acquirer browns out to 30s latency and a client retries every 5s. One payment is now the original's connection plus ~6 blocked duplicates; at Npgsql's default pool of 100, roughly fourteen concurrent slow payments exhaust the pool and *everything* fails, reads and readiness checks included. A partner brownout becomes a full outage. The alternative (commit the reservation first, then work) frees the connections but needs lease/expiry logic to recover keys

stranded by a crash. At this scale the simpler, always-consistent option wins; at real throughput the retry-storm multiplier is what forces the other design.

Failures don't burn an idempotency key. If the work throws, the reservation rolls back with it, so only responses actually returned are replayable and a client can retry a failed call with the same key. The alternative — storing 4xx/5xx outcomes — makes a transient bug permanently replayable.

Replays return status and body, not the full header set. A replayed create is a 201 with the identical body and an `Idempotency-Replayed: true` header, but no `Location`. This is how Stripe replays, and storing whole header sets to reproduce one header didn't earn its keep.

Polling with SKIP LOCKED, not LISTEN/NOTIFY. Simpler, and the row lock is the mutual exclusion, so N workers need no distributed lock. The cost is idle settlement latency $\approx$ the 2s poll interval and a small constant query load. LISTEN/NOTIFY would cut the latency; at exercise volumes it isn't worth the connection management.

The claim doubles as a lease. Claiming stamps `next_attempt_at` into the future, and the poll selects `Pending` *OR* `Processing` rows that are due — so a worker that crashes mid-job has its work reclaimed when the lease lapses rather than stranding the job forever. No schema change needed, and `attempt_count` increments at claim time so a crash loop still counts toward dead-lettering. Redelivery is the tradeoff: every job must be treated as at-least-once.

Settlement is idempotent at the acquirer, keyed by the job id. The lease means a job *will* sometimes be delivered twice. Settlement is the one call that moves real money, so `SettleAsync` takes an idempotency key and the worker passes the settlement job's id — stable across every attempt and every redelivery. An ambiguous timeout (did it settle, or not?) is then resolved by the acquirer replaying the original settlement rather than performing a second one. A fresh key per attempt would make every retry a new settlement, which is exactly the failure the retry logic exists to survive. Note a broker would **not** have fixed this — SQS and Rabbit are at-least-once too; only the key is load-bearing. Transient failures deliberately release the key so the next attempt genuinely re-tries; only a completed settlement is replayable — the same rule the API's own store follows, for the same reason. The fake gateway implements this contract rather than stubbing it, so the retry and redelivery paths are tested against something that actually dedupes.

Business metrics count after commit, and never on replay. A client retrying a POST five times moves RED by five and `payments_created_total` by one. Counting inside the work would have inflated both the retry and the rollback cases, and a counter named `payments_created` that doesn't mean payments created is worse than no counter.

/healthz checks nothing on purpose. A liveness probe that depends on Postgres has Kubernetes kill every API pod during a database blip, turning a recoverable incident into an outage. Readiness (`/readyz`) is where the dependency check belongs.

A simulated gateway. Keeps PCI scope at zero and makes the failure paths demonstrable (configurable decline/failure rates, a deterministic `-declined` token). No real acquirer integration is attempted.

Assumptions

Where the brief was ambiguous, this is what was assumed and why:

- **Card data arrives pre-tokenized**, as a client-side tokenization SDK would produce: an opaque token plus display-safe metadata. The platform holds `cardLast4`, `cardBrand` and the token; no PAN/CVV field exists anywhere in the domain, the DB or the API. That's structural, not a redaction rule someone has to remember. Read the other way — raw card data hitting the API — the build would need field-level encryption, log scrubbing and a vault, and the exercise would have become a PCI exercise.
- **Merchant identity is trusted from a header.** This is an auth stub, not auth; the repository-level scoping it feeds is real and is what production auth would plug into.

- **One currency per payment, no FX.** Amounts are integer minor units — never floats.
- **Full-amount capture and refund only.** Partial operations change the aggregate's shape (amounts per operation rather than a single status) and were cut for scope, not missed.
- **Authorization is synchronous at create time** (auth-then-capture); settlement is the async leg. An async-auth reading would have moved the worker earlier in the lifecycle but exercised the same machinery.
- **Refunds don't call the gateway.** The state change and audit trail are what's being demonstrated; a real refund is a gateway call with its own idempotency and failure modes.
- **Single-region, single database.** No sharding, no read replicas.

Testing

Domain unit tests assert the full 36-pair transition matrix, legal and illegal — fast, no I/O, and the highest-value tests in a payments system. **Integration tests** run the real API against a throwaway Postgres via Testcontainers, because SKIP LOCKED, unique-index contention and xmin concurrency are Postgres behavior — an in-memory provider would prove nothing about the only things worth testing at that level. The two concurrency tests fire eight racing requests at one idempotency key and assert exactly one did the work. **Frontend tests** (Vitest + Testing Library) cover the money-adjacent logic rather than pixels: the key one drives the real detail component through a mocked network failure and asserts on the idempotency keys that actually went over the wire — a retry of the same attempt reuses the key, a new attempt gets a fresh one.

Deliberately skipped: contract tests (one consumer, and it lives in this repo), load tests (they'd earn their keep once the connection-per-in-flight-payment tradeoff above starts to bite), and browser E2E (the component tests exercise the behavior worth protecting without a browser to babysit).

Production considerations

What separates this from a shippable system, named rather than left to be found:

Authorization can be orphaned at the acquirer — the one known correctness gap. `create` calls the gateway *inside* the idempotency transaction. If that transaction rolls back after the acquirer approved — a commit failure, a crash — the authorization exists on the customer's card but no payment row does, and because a failure deliberately doesn't burn the key, the client's retry authorizes a second time. The customer sees two holds. The fix is the same shape as the settlement fix above: pass a gateway idempotency key derived from the client's `Idempotency-Key`, so the acquirer collapses the retry onto the original authorization. I ran out of time to close it; it's first in the future-work list because it's a correctness gap, not a feature.

Authentication and authorization. Replace the header with API keys or OAuth2 client-credentials per merchant, resolved in middleware into the same merchant scoping the queries already apply; add per-merchant rate limiting. Because scoping is enforced at the repository boundary, this changes *where identity comes from*, not every query.

Migrations are a deploy step, not app startup. They auto-apply here only in Development. In production this is a pipeline-gated migration job, so a rolling deploy can't have two versions racing to migrate.

Secrets and TLS. The compose password and connection string are plain config because they're local-only; real deployment pulls them from a secrets manager. Everything speaks plain HTTP because everything is localhost; in production TLS terminates at the ingress and nothing in the code assumes a scheme.

Gateway calls have no timeout budget or circuit breaker. The in-process fake hides it, but a real acquirer is a network partner: production wraps `IPaymentGateway` in per-call timeouts and a circuit breaker (Polly), and the idempotency keys on both sides are what make those timeouts safe to take.

Unbounded growth. Idempotency keys are never swept and terminal settlement jobs are never archived — both tables grow forever. Keys need a TTL job (24h is typical); the queue’s stats query already filters to live statuses so the metric doesn’t become the queue’s biggest customer, but archival is what keeps the table itself honest.

Observability wiring. Logs are JSON on stdout, metrics are Prometheus text at `/metrics`, and traces register only when `OTEL_EXPORTER_OTLP_ENDPOINT` is set — the observability profile lights all three without touching application code, which is the property that matters. Two rules held throughout: metric label values are bounded (enums and route templates, never ids or gateway error strings — unbounded labels are how a metrics backend gets taken down by its own instrumentation), and successful probe/scrape traffic is dropped from logs and traces (left at `Information`, probes were 77% of the local log stream) while anything that throws or 5xxs is still an `Error`, whatever the path. One trace gap is deliberate and worth naming: the worker’s spans start *new* traces rather than continuing the capture’s, because W3C trace context is not persisted across the queue. The correlation id is — that’s the join key between the two traces — and carrying `traceparent` on the job row the same way would stitch them into one. In production the custom `X-Correlation-Id` would fold into `traceparent` rather than running two tracing systems side by side, and `/metrics` would be scraped from inside the network rather than publicly proxied as the demo does for convenience.

Dead-letter runbook. A dead job leaves the payment `Captured` and visible in the `settlement_dead_jobs` gauge. There is deliberately no automatic replay — an operator should understand *why* five attempts failed before money moves. Alert on the gauge, not the counter: counters reset on restart and a real backlog would vanish with one.

Horizontal scaling. The API is stateless and everything that arbitrates a race does it in the database — the idempotency unique index, `xmin` on payments, `SKIP LOCKED` on the queue — so `N` API instances and `N` workers are safe with no coordination service. (Even the demo seeder serializes itself with an advisory lock so two instances can’t double-seed.) The known ceiling is the idempotency design’s connection pinning, described in Tradeoffs.

Future improvements

Genuine next steps, in the order I’d do them:

1. **Gateway-side idempotency key on authorization** — closes the orphaned-authorization window, the only known correctness gap; that’s why it’s first.
2. **Split Refund into Void and Refund** — reversing before settlement is an auth reversal (money never moved); after, a true refund (money moves back). Different gateway calls, different failure modes; the state machine should name them. Related: authorizations should expire — real card auths die in about seven days, and abandoned `Authorized` payments currently sit forever.
3. **Retention** — idempotency-key TTL sweep and settlement-job archival; both tables grow unbounded.
4. **Reconciliation against the acquirer** — a daily job comparing settled payments to the gateway’s report. In a real system this, not the retry logic, is what catches money that actually went missing.

Cut for scope — worth doing, but only when the need arrives:

5. **Webhooks** — the platform records everything and tells no one; the fix is the outbox pattern a second time (event rows, a dispatcher with retries and dead-lettering, delivery keyed by event id). That `settlement_jobs` is a specialization of a pattern that generalizes this cleanly is the strongest evidence the original choice was right.
6. **Partial and multiple captures/refunds** — the real shape of the domain; the aggregate would carry amounts per operation rather than a single status.
7. **LISTEN/NOTIFY, then a broker relay** — removes the polling latency floor once it matters, then the outbox → broker migration when settlement needs to scale independently or fan out to other consumers.